

System Scanner Pro

AI-Powered Distributed Endpoint Monitoring & Remote Scanning Platform

Technical Documentation Package | v3.1

Handover Document

Everything a successor team needs to take ownership safely

Project	System Scanner Pro (v3.1)
Document	Handover Document
Package date	August 21, 2026
Prepared by	Project Team - Kailasanathan (Team Leader), Saagr, Mega
Audience	Developers, administrators, testers and new team members
Status	Final

This document is part of a nine-volume documentation package generated from the actual project source code. All commands, file paths, endpoints and behaviours described here were verified against the repository at documentation time.

Table of Contents

1. Purpose & How to Use This Handover	3
2. Project Summary Card	3
3. People & Contacts	3
4. Source Code & Version Control	3
5. Environments Overview	3
6. Access & Credentials Matrix	4
7. Credentials Policy & Rotation Rules	4
8. CRITICAL - Committed Secrets Cleanup	4
9. Architecture Recap (One Page)	5
10. Deployment Topology Summary	5
11. Operating Rhythms (Daily/Weekly/Monthly)	5
12. Runbook Pointers	5
13. Known Issues - CRITICAL	5
14. Known Issues - HIGH	5
15. Known Issues - MEDIUM / LOW	6
16. Technical Debt Register	6
17. Pending Work / Roadmap Inputs	6
18. Test Assets Status	6
19. Documentation Package Map	7
20. Data Ownership & Backup Responsibilities	7
21. Third-Party / License Status	7
22. Support & Escalation Path	7
23. Acceptance Checklist for the New Maintainer	7
24. First 48 Hours Action Plan	8
25. Sign-off & Transfer of Responsibility	8
26. Appendix - Repository Facts Cheat-Sheet	8

1. Purpose & How to Use This Handover

This volume transfers operational ownership. It records who knows what, where access lives, what is fragile, and what remains unfinished. Items marked [TO BE PROVIDED] must be filled by the current owners before final sign-off; items marked [PROVIDED SEPARATELY] are delivered through the secure channel your organisation uses for secrets - never via this document or git.

2. Project Summary Card

Field	Value
Product	System Scanner Pro - AI-powered distributed endpoint monitoring & remote scanning platform
Version	v3.1 (README canonical; banner v2.1 / agent v1.6.1 / older docs say v3.0 - unify as first cleanup task)
Core stack	Django 5 + DRF + Channels/Daphne + APScheduler + ReportLab/openpyxl; Python agent (stdlib + websockets + watchdog); PostgreSQL/Supabase or SQLite
Deployment modes	VPS (nginx+systemd+Daphne) recommended; Vercel serverless supported (REST-only realtime)
Repository	https://github.com/shinecraftinternship-kms/AdminClient.git
Docs package	This nine-volume set in PROJECT_DOCUMENTATION_PACKAGE/
Team at handover	Kailasanathan (Leader), Saagr, Mega

Table 1 - Summary card

3. People & Contacts

Name	Role	Owns	Contact
Kailasanathan	Team Leader	Auth, agent, WebSocket layer, API, integration	[TO BE PROVIDED]
Saagr	Developer	Assets, licenses, compliance, alerts	[TO BE PROVIDED]
Mega	Developer	Intelligence/reports, notifications/audit, health scoring	[TO BE PROVIDED]
Project guide/supervisor	Oversight	Approvals	[TO BE PROVIDED]
Hosting owner	Ops	VPS/Vercel/Supabase billing and root access	[TO BE PROVIDED]

Table 2 - Contacts

4. Source Code & Version Control

Remote - <https://github.com/shinecraftinternship-kms/AdminClient.git> - single branch main.

State at handover - Working tree clean; latest commits cover the Supabase switch, richer scan payloads and rebuilt client binary.

Branching convention - None enforced to date - adopt feature-branch + PR flow going forward.

CI/CD - Only `.github/workflows/supabase-register.yml` (hourly discovery keep-alive). No test/deploy pipelines yet.

Binaries in repo - `client_scanner.exe/.zip` are committed for distribution convenience; consider GitHub Releases instead.

5. Environments Overview

Environment	Where	Database	Notes
Local dev	Developer machines	SQLite (admin/data/scanner.db)	python admin/main.py --port 8000
Staging/demo (VPS)	[TO BE PROVIDED - host]	SQLite or Postgres [TO BE PROVIDED]	install.sh deployed; TLS via Certbot
Production VPS	[TO BE PROVIDED]	[TO BE PROVIDED]	system-scanner-admin.service
Vercel preview/prod	[TO BE PROVIDED - *.vercel.app]	Supabase Postgres (pooler 6543)	REST-only realtime
Supabase project	[TO BE PROVIDED - ref/region]	Managed Postgres	Holds app schema + server_registry discovery row

Table 3 - Environment inventory

6. Access & Credentials Matrix

Never store these in the repository

All values below must be transferred via your password manager / secrets channel.

Asset	Location	Holder	Transfer status
GitHub repo admin	github.com	[TO BE PROVIDED]	[PROVIDED SEPARATELY]
VPS root/sudo user	Host provider	[TO BE PROVIDED]	[PROVIDED SEPARATELY]
Domain/DNS control	Registrar	[TO BE PROVIDED]	[PROVIDED SEPARATELY]
Supabase dashboard	supabase.com	[TO BE PROVIDED]	[PROVIDED SEPARATELY]
SUPABASE_SERVICE_KEY	Server .env only	(server process)	[PROVIDED SEPARATELY]
DATABASE_URL	Server/Vercel env	(server process)	[PROVIDED SEPARATELY]
DJANGO_SECRET_KEY	.env per environment	(server process)	[PROVIDED SEPARATELY]
Default admin login	First-run seed	Must be changed on takeover	Rotate immediately (see checklist)
RESET_TOKEN override	api/index.py env	(serverless)	[PROVIDED SEPARATELY]

Table 4 - Credential transfer matrix

7. Credentials Policy & Rotation Rules

- Rotate DJANGO_SECRET_KEY only during a maintenance window: it invalidates sessions, signed cookies and JWTs (users re-login).
- Rotate Supabase service key by creating a new one, updating server .env + GitHub Actions secret, then revoking the old.
- Rotate DATABASE_URL password in Supabase -> update Vercel env -> redeploy.
- Agent secrets (AgentSecret rows) rotate per-device from monitoring UI/API; agents re-handshake automatically.
- API keys (sk_...) revoke instantly via deactivation; issue replacements before revoking active integrations.

8. CRITICAL - Committed Secrets Cleanup

Action required before public exposure

The repository currently tracks **.env.template** containing real-looking values: a Supabase database URL including its password, a service-role key and a JWT secret. Treat them as compromised: rotate every one of them now (they are listed in section 7), replace the template with placeholder text, and remove it from tracking. If the repo may be made public later, also rewrite history (git filter-repo) or archive the repo.

Secondary hygiene: client_crash.log is tracked and should be untracked; large binaries belong in Releases rather than git.

9. Architecture Recap (One Page)

Four Django apps compose the server: scanner_api (identity, clients, scans, org/assets, auth), monitoring (telemetry ingest, health/anomaly/predictive engines, scheduler, agent WebSocket), maintenance (records/warranty/downtime/licenses/compliance) and intelligence (alerts/rules/notifications/reports/audit). Agents register with key+fingerprint, await approval, then stream heartbeats/metrics/events and run scheduled or pushed scans. Browsers consume pages + WebSocket dashboards. Full diagrams: volume 02.

10. Deployment Topology Summary

VPS path - nginx(TLS) -> 127.0.0.1:8000 systemd service running admin/main.py --asgi --domain -> SQLite data dir or DATABASE_URL; UDP 45000 open on LAN.

Serverless path - vercel.json -> api/index.py (gunicorn WSGI, whitenoise, signed cookies) -> Supabase pooler; hourly Action refreshes discovery row.

Client rollout - build_client.py produces dist/client_scanner.exe; distribute exe or connect URL.

11. Operating Rhythms (Daily/Weekly/Monthly)

Cadence	Tasks
Daily	Glance executive dashboard; clear critical alerts; confirm backup job success
Weekly	Review pending approvals queue; scan-diff surprises; license expiry horizon; disk-full ETAs
Monthly	Patch host + dependencies; restore-drill one backup; audit-log spot review; prune stale devices
Quarterly	Rotate credentials per policy; review anomaly false-positive tuning; update this handover

Table 5 - Operations calendar

12. Runbook Pointers

Step-by-step procedures live elsewhere in this package: install/upgrade/backup in volume 03; daily UI operations in volume 07; testing routines in volume 08; deep module behaviour and triage in volume 05. Keep those volumes next to this one.

13. Known Issues - CRITICAL

#	Issue	Impact	Recommended action
C1	.env.template tracked with real-looking credentials	Secret leakage if repo shared/public	Rotate all three values; replace template placeholders; untrack file (section 6)
C2	api/index.py / __reset/ has hardcoded fallback token	Data wipe if exposed without override	Always set RESET_TOKEN env on any shared deployment
C3	DEBUG defaults True; ALLOWED_HOSTS defaults *	Information disclosure / host-header abuse in misconfigured prod	Set DJANGO_DEBUG=False + explicit hosts everywhere
C4	SECRET_KEY insecure fallback in settings	Forgeable cookies if unset	Set strong DJANGO_SECRET_KEY (installer warns)

Table 6 - Critical issues

14. Known Issues - HIGH

#	Issue	Impact	Recommended action
H1	CORS_ALLOW_ALL_ORIGINS=True	Any site can call API with victim cookies	Restrict origins list
H2	CsrfExemptSessionAuthentication on DRF + csrf_exempt views	CSRF surface on cookie-authed endpoints	Require JWT/API-key for non-browser clients; re-enable CSRF where feasible
H3	Password reset sends no email	Self-service recovery broken	Add SMTP backend + send_mail in ForgotPasswordRequestView
H4	DEFAULT_PERMISSION_CLASSES empty	Permission classes opt-in only; easy to forget on new endpoints	Set sensible default + audit views
H5	In-memory channel layer	No horizontal WS scale	Swap to Redis layer when scaling out

Table 7 - High issues

15. Known Issues - MEDIUM / LOW

#	Issue	Note
M1	Version strings inconsistent (3.1/2.1/1.6.1/3.0)	Unify display versions
M2	client_crash.log tracked	Untrack; add to .gitignore pattern already covering logs
M3	Binaries committed	Move to Releases/LFS
M4	MFA fields unused	Wire TOTP or drop fields to avoid false confidence
L1	MANUAL_TESTING cases all Pending	Execute per volume 08 plan
L2	RateLimiter in-memory only	Per-process limits; fine single-node

Table 8 - Medium/low issues

16. Technical Debt Register

- views.py (~4,100 lines) deserves splitting into modules per cluster (auth/clients/scans/org/assets).
- Tests cover registration semantics well; auth/JWT/report paths rely on manual cases - automate next.
- Frontend JS lacks linting/tests; consider eslint + minimal Playwright smoke suite.
- No CI: add GitHub Actions running ruff + pytest/Django tests on PRs.

17. Pending Work / Roadmap Inputs

- Email delivery (reset + notifications digest) - top user-visible gap.
- TOTP MFA completing existing model fields.
- Redis channel layer + multi-worker deployment guide.
- Linux/macOS metric parity in agent collectors.
- Automated CI pipeline incl. migrations check and packaging build.

18. Test Assets Status

- Automated: 4 unit test modules under admin/scanner_api/tests/ (registration/approval semantics heavily covered).
- Manual: MANUAL_TESTING.txt/.xlsx defines 30 essential cases across 17 modules, owned by Kailasanathan/Saagr/Mega - ALL currently marked '[] Pending' and therefore reported as [TO BE TESTED] throughout this package.
- No results may be claimed until executed and recorded (volume 08 provides the protocol).

19. Documentation Package Map

Volume	Audience
01 Project Documentation	Stakeholders/evaluators
02 System Design & Diagrams	Architects/new devs
03 Installation & Configuration	Ops/first setup
04 Codebase Documentation	Developers
05 Explanation & Knowledge Transfer	Everyone (module briefs)
06 Handover Document	Successor team (this volume)
07 User & Admin Manual	End users/administrators
08 Testing & Maintenance Guide	QA/ops
09 Complete Human Knowledge Transfer Guide	New joiners

Table 9 - Package contents

20. Data Ownership & Backup Responsibilities

- Owner of record for production data: [TO BE PROVIDED].
- Backups: follow volume 03 section 'Backup Procedures'; verify restores monthly (calendar section 11).
- Retention: intelligence RetentionPolicy rows define alert/notification pruning; DB dumps retained per org policy [TO BE PROVIDED].

21. Third-Party / License Status

- All runtime dependencies are permissive-licensed (BSD/MIT/Apache-style) - no copyleft obligations introduced by requirements.txt.
- Frontend assets load from public CDNs (Bootstrap/Icons/Chart.js) - pin versions when air-gapping.
- Project's own license: [TO BE PROVIDED].

22. Support & Escalation Path

Level	Who	When
L1	Internal IT using volumes 07/08 checklists	Routine questions, approvals, exports
L2	Successor dev team	Bugs, config, deploys
L3	Original authors during warranty window [TO BE PROVIDED]	Architecture-level changes
Vendor	Supabase/Vercel support portals	Platform incidents

Table 10 - Escalation ladder

23. Acceptance Checklist for the New Maintainer

1. Clone repo; run local stack; log in with YOUR OWN created superuser.
2. Change/retire seeded default admin credentials on every environment you touch.
3. Confirm .env values present for each environment (section 5) via secure channel.
4. Rotate items C1-C4 (section 9) or verify predecessor did.
5. Run python admin/manage.py test scanner_api locally - expect green suite.
6. Deploy a staging instance via volume 03; enroll one device end-to-end.

7. Execute MANUAL_TESTING Modules 1-3 and record outcomes.
8. Schedule credential rotation calendar per section 7.

24. First 48 Hours Action Plan

Hour	Action
0-2	Read volumes 01+02; skim 04 section map
2-6	Section 13 checklist steps 1-4 (access + rotations)
6-12	Local stack up; unit tests green; explore dashboard with sample agent
12-24	Staging deploy via install.sh; TLS; first approval flow
24-48	Backup + restore drill; execute first 10 manual test cases; open issues for gaps found

Table 11 - Onboarding sprint

25. Sign-off & Transfer of Responsibility

Role	Name	Signature	Date
Outgoing lead	Kailasanathan		
Outgoing developer	Saagr		
Outgoing developer	Mega		
Incoming owner	[TO BE PROVIDED]		
Supervisor	[TO BE PROVIDED]		

Table 12 - Handover signatures

26. Appendix - Repository Facts Cheat-Sheet

```
Repo      : https://github.com/shinecraftinternship-kms/AdminClient.git
Branches  : main
Server cmd: python admin/main.py [--port N] [--asgi] [--domain H]
Manage    : python admin/manage.py <migrate|createsuperuser|test|...>
Agent     : python client/main.py "<connect-url>" | dist/client_scanner.exe
Build exe : python build_client.py
VPS       : sudo DOMAIN=<host> bash deploy/install.sh
Service   : systemctl {status|restart} system-scanner-admin
Logs      : journalctl -u system-scanner-admin -f ; admin/local_server.log
DB        : SQLite admin/data/scanner.db | DATABASE_URL (Supabase 6543 pooler)
```